

Guió oral complet imprimible

Classe magistral de 20 minuts: Disseny de ports de sortida en arquitectura hexagonal

Temps total previst: 20:00.

Diapositiva 1. Portada - 1:45

Bon dia, membres del tribunal. [pausa breu, mirada al tribunal]

La classe que presento porta per títol Disseny de ports de sortida en arquitectura hexagonal, i se centra en el desacoblament entre casos d'ús, domini i APIs externes. [assenyalar el títol]

Abans d'entrar en el contingut tècnic, voldria fer una precisió de context. A la documentació del tribunal he incorporat un informe acreditatiu de la meva activitat de formació i consultoria especialitzada per a l'AGAUR, l'Agència de Gestió d'Ajuts Universitaris i de Recerca. [pausa breu]

Ho explico al principi perquè aquesta sessió està plantejada deliberadament a partir d'una doble vinculació. D'una banda, és una classe situada dins l'assignatura Disseny de Sistemes d'Informació, que imparteixo juntament amb el Dr. Josep Roure. De l'altra, parteix d'un cas professional real en l'àmbit de l'administració pública catalana. [mirada al tribunal]

La intenció no és revelar cap detall intern, ni cap endpoint, ni cap dada sensible. La intenció és docent: mostrar com conceptes que treballem a l'aula, com ports, adaptadors, inversió de dependències i separació entre domini, aplicació i infraestructura, apareixen també en problemes reals d'evolució de sistemes d'informació. [pausa]

En vint minuts no seria rigorós intentar explicar tota l'arquitectura hexagonal. Per això em centraré en una decisió concreta: com dissenyar un port de sortida quan un cas d'ús necessita informació que es troba fora de l'aplicació.

Diapositiva 2. On som dins l'assignatura - 1:10

La sessió se situa dins Disseny de Sistemes d'Informació, una assignatura obligatòria de tercer curs, de 6 ECTS. [assenyalar la part esquerra]

El pla docent de l'assignatura explica que s'hi treballa una visió global d'arquitectures de sistemes d'informació: arquitectures per capes, arquitectures clean o hexagonals, i arquitectures basades en microserveis. Dins el bloc d'arquitectures clean o hexagonals hi apareixen explícitament ports i adaptadors, mapping entre capes i organització en mòduls. [to calmat, explicatiu]

Per tant, aquesta no és una sessió afegida artificialment al temari. És una classe versemblant dins l'assignatura. [pausa breu]

El que farem avui és acotar molt el focus. No analitzarem tota l'arquitectura hexagonal. Ens centrarem en els ports de sortida: aquells ports que permeten que l'aplicació obtingui o persisteixi informació mitjançant sistemes externs, sense que el cas d'ús quedi lligat a una tecnologia concreta.

Diapositiva 3. Objectiu de la sessió - 0:55

Abans d'entrar en el cas, deixo formulat l'objectiu de la sessió. [pausa breu]

El principi de treball és aquest: el cas d'ús necessita una capacitat, no una tecnologia. [mirar el tribunal]

Per tant, el que aprendrem és a fer tres operacions: identificar una necessitat externa del cas d'ús, expressar-la com a port de sortida i implementar-la mitjançant adaptadors substituïbles. [assenyalar les tres caixes]

No desenvoluparé encara tota l'abstracció. Primer veurem el problema en un cas real d'administració pública, i a partir d'aquí formularem la decisió arquitectònica. [pausa breu]

Diapositiva 4. Avaluació econòmica d'una beca - 1:50

Abans de parlar d'arquitectura de software, cal entendre mínimament el procediment administratiu. [to més lent]

En un procediment de beca, una sol·licitud dona lloc a un expedient. Aquest expedient passa per diferents fases: requisits generals, requisits econòmics, revisió acadèmica i resolució. [assenyalar la seqüència de la dreta]

La part que ens interessa és l'avaluació econòmica. En aquest punt es comproven aspectes com la renda familiar i el patrimoni. Quan no s'acredita el compliment dels requisits econòmics, l'expedient no obté una valoració favorable en aquesta fase. [pausa breu]

La idea docent important és que aquí encara no hem parlat de SOAP, ni de REST, ni de PICA, ni de cap base de dades. Estem parlant de la necessitat funcional del procediment: per poder aplicar uns criteris econòmics, el sistema necessita dades fiables sobre renda i patrimoni.

Aquest pas és essencial. Si comencem directament per la tecnologia, correm el risc de construir el cas d'ús al voltant de la integració. En canvi, si comencem pel procediment, podem distingir entre la decisió administrativa que volem modelar i el mecanisme tècnic que ens proporciona la informació.

Diapositiva 5. Quina informació externa necessita el procediment? - 2:00

Per avaluar renda i patrimoni, el procediment pot necessitar informació que no neix dins l'aplicació. [assenyalar la columna de necessitats]

En termes funcionals, podem parlar de renda familiar, patrimoni i béns immobles. En termes de fonts administratives, apareixen organismes i plataformes com l'AEAT, el Cadastre i PICA. [assenyalar la columna de fonts]

PICA és la Plataforma d'Integració i Col·laboració Administrativa de la Generalitat de Catalunya. Per a aquesta classe, no cal entrar en tots els detalls tècnics de PICA. El que ens interessa és que actua com a mecanisme corporatiu d'interoperabilitat administrativa.

La distinció clau és aquesta: el procediment necessita informació administrativa; no necessita, en el seu llenguatge propi, una API concreta. [pausa llarga; mirar el tribunal]

Això no vol dir que l'API concreta no sigui important. Ho és, i molt. Però pertany a un altre nivell de decisió. El cas d'ús ha de poder dir: **necessito informació patrimonial avaluable**. L'adaptador ja resoldrà si aquesta informació arriba per SOAP, REST, PICA, un certificat, una base de dades corporativa o un mecanisme futur.

Quan aquesta separació no es respecta, apareix el problema arquitectònic que veurem ara.

Diapositiva 6. Dependència directa entre procediment i integració - 2:10

Imaginem una situació en què el cas d'ús queda vinculat directament a la cadena tècnica: SOAP/XML, PICA, AEAT o Cadastre. [assenyalar la cadena central]

El problema no és consumir PICA. Ho remarco perquè és important. PICA, en aquest context, és una plataforma corporativa d'interoperabilitat. El problema arquitectònic apareix quan la lògica del procediment administratiu depèn directament dels detalls de la integració: DTOs externs, estructures XML, clients SOAP, codis d'error tècnics o convencions d'una API concreta. [pausa breu]

Quan això passa, l'evolució tecnològica travessa la frontera del cas d'ús. Si un servei evoluciona, si canvia el contracte tècnic, si cal passar de SOAP a REST, o si apareix un altre mecanisme corporatiu, el canvi pot impactar codi que hauria d'estar expressant criteris del procediment.

Des del punt de vista de l'enginyeria de software, això incrementa tres riscos. Primer, acoblament tecnològic: el cas d'ús coneix massa detalls externs. Segon, fragilitat del manteniment: canvis d'infraestructura obliguen a revisar lògica d'aplicació. I tercer, risc de continuïtat operativa: una evolució tècnica pot afectar un procediment que hauria de romandre estable. [assenyalar les tres caixes]

La pregunta docent, per tant, és: com podem permetre que el cas d'ús necessiti dades externes, però sense dependre directament de la tecnologia que les proporciona?

La resposta és introduir un port de sortida.

Diapositiva 7. Port de sortida i adaptador - 1:30

Un port de sortida és un contracte definit des de l'aplicació. [pausa breu]

El cas d'ús o el servei d'aplicació diu: **necessito aquesta capacitat**. Per exemple: obtenir informació patrimonial, recuperar un expedient, desar una transacció o consultar el preu actual d'una acció. El port expressa aquest què. [assenyalar el port]

L'adaptador, en canvi, resol el **com**. És la peça que coneix la tecnologia concreta: SOAP, REST, JDBC, JPA, un client HTTP, una estructura XML, JSON o el mecanisme que sigui necessari. [assenyalar adaptador i sistema extern]

Aquí convé distingir dues fletxes. Si parlem de flux d'execució, el servei pot cridar el port i arribar finalment a l'adaptador. Però si parlem de dependència de codi, l'adaptador depèn del port, perquè l'adaptador implementa la interfície definida per l'aplicació. [mirada al tribunal]

Aquesta és la inversió de dependències: el nucli no depèn de la implementació concreta; la implementació concreta depèn del contracte que defineix el nucli.

Diapositiva 8. Port de sortida per a informació patrimonial - 2:05

Portem aquesta idea al cas administratiu. [assenyalar el diagrama]

El servei d'aplicació que avalua econòmicament una beca no hauria de conèixer directament els detalls de PICA, SOAP/XML, REST o JDBC. Hauria de dependre d'un contracte expressat en el llenguatge de l'aplicació. Per exemple: obtenir informació patrimonial, recuperar dades de renda o carregar l'expedient administratiu.

En el diagrama, el nucli de l'aplicació conté el servei d'aplicació i el model de domini. Al voltant hi ha ports: un port d'entrada per invocar el cas d'ús i ports de sortida per obtenir o persistir informació. Fora del nucli hi ha els adaptadors: el controlador REST, l'adaptador JDBC, l'adaptador d'interoperabilitat amb PICA o qualsevol altre mecanisme concret.

El punt tècnic més important és que l'adaptador de PICA o l'adaptador JDBC no són el centre de l'arquitectura. Són substituïbles. El que és estable és el contracte que el cas d'ús necessita. [pausa breu]

Aquesta distinció queda reflectida en la direcció de les fletxes. Quan indiquen *implemented by*, les fletxes van de l'adaptador cap al port. El servei usa el port. L'adaptador implementa el port. I el domini no coneix ni l'adaptador, ni PICA, ni SOAP, ni REST.

Aquesta mateixa estructura la podem transferir ara a un domini financer docent: HexaStock.

Diapositiva 9. Transferència a domini financer - 1:45

HexaStock ens permet treballar el mateix patró en un domini diferent: la venda d'accions. [assenyalar el títol]

Aquí el cas d'ús és vendre accions d'una cartera. Per fer-ho correctament, el servei d'aplicació necessita el preu actual del ticker. Però el cas d'ús no hauria de dependre de Finnhub, Alpha Vantage, tokens, endpoints, JSON, clients HTTP o disponibilitat d'internet. [pausa breu]

El port de sortida és `StockPriceProviderPort`. Aquest port expressa la capacitat que l'aplicació necessita: obtenir el preu d'una acció. Després podem tenir diferents adaptadors que implementen aquest port: `FinnhubStockPriceAdapter`, `AlphaVantageStockPriceAdapter` o un adaptador mock per a proves i demos controlades.

El patró és idèntic al cas AGAUR. En el cas administratiu, el procediment necessita informació patrimonial, no una tecnologia concreta. En HexaStock, el cas d'ús necessita un preu actual, no un proveïdor concret. [mirada al tribunal]

Per tant, el missatge docent és transferible: primer identifiquem la necessitat funcional; després definim el port; finalment implementem adaptadors.

Diapositiva 10. Flux i codi essencial - 1:45

Aquesta diapositiva mostra el flux temporal del cas d'ús de venda. [assenyalar el diagrama de seqüència]

Convé aclarir que, en un diagrama de seqüència, les fletxes representen ordre d'execució, no dependències de codi. Això és perfectament compatible amb la inversió de dependències que acabem d'explicar.

El client o controlador invoca el port d'entrada, `PortfolioStockOperationsUseCase`. La implementació és el servei d'aplicació, `PortfolioStockOperationsService`. Aquest servei recupera el portfolio mitjançant un port de persistència, consulta el preu mitjançant `StockPriceProviderPort`, i després crida el domini: `portfolio.sell(ticker, quantity, price)`. [pausa breu]

La separació de responsabilitats és molt important. El servei coordina. El port obté informació. L'adaptador integra amb la infraestructura. Però la decisió de domini, la venda i el càlcul del resultat, no es resolen ni al controlador ni a l'adaptador de preus. Es resolen al model de domini, dins l'agregat `Portfolio`.

Dit de manera sintètica: l'adaptador pot saber com obtenir un preu; no hauria de decidir com es ven una acció dins la cartera. [pausa]

Diapositiva 11. Demo - 1:45

La demo, en aquesta classe, no pretén impressionar per complexitat tècnica. Té una funció docent molt concreta: mostrar que podem mantenir el mateix cas d'ús, el mateix servei d'aplicació i el mateix domini, canviant només l'adaptador. [assenyalar les tres caixes]

En `HexaStock`, això es pot veure amb perfils diferents: un adaptador `Finnhub`, un adaptador `Alpha Vantage` o un adaptador `mock`. El port que veu el servei és el mateix: `StockPriceProviderPort`. La implementació concreta canvia segons la configuració.

Aquest punt és especialment rellevant en docència. Permet fer proves sense dependre d'internet, sense claus reals, sense disponibilitat d'un proveïdor extern i sense contaminar el domini amb detalls tècnics. [mirada al tribunal]

El missatge de la demo és el mateix que hem treballat des del principi: canvia la infraestructura; el cas d'ús i el domini romanen estables.

Això no vol dir que l'arquitectura elimini el canvi. El canvi continua existint. El que fa una bona arquitectura és localitzar-lo.

Diapositiva 12. Agraïment - 1:20

Per tancar, voldria recuperar aquesta idea final. [pausa breu]

L'arquitectura no elimina la dependència del món exterior. Les aplicacions reals necessiten bases de dades, APIs, plataformes corporatives, proveïdors externs i mecanismes d'integració. El que fa l'arquitectura hexagonal és convertir aquesta dependència en una frontera explícita, substituïble i comprovable.

Quan canvia un proveïdor, una API o una tecnologia, volem canviar l'adaptador. No volem reescriure el cas d'ús ni deformar el model de domini. [pausa]

Vull expressar el meu agraïment al TecnoCampus per un entorn que m'encoratja a créixer, tant en la docència com en l'àmbit acadèmic, al costat d'un equip del qual aprenc i amb qui comparteixo coneixement, responsabilitat i compromís amb la formació universitària. [pausa breu; mirar el tribunal]

També vull fer un agraïment especial al Dr. Josep Roure, company professor del TecnoCampus, amb qui he tingut l'oportunitat d'aprendre sobre arquitectura de software, metodologies actives d'aprenentatge i treball en equip. [mirada al tribunal]

Moltes gràcies.